

2026-08-19-6th-forced-logout

6th Forced-Logout Incident — 2026-08-19

Revision: 2 **Last modified:** 2026-08-19T14:14:05Z **Tracker:** BOB-125 (to be minted) **Verdict (§11.4.6):** ROOT CAUSE STILL UNCONFIRMED — Phase 1 attribution attempt live **REV-2 correction:** initial reading claimed “different mechanism than BOB-124 because I stayed connected” — WRONG. tmux tmx-boba-5085.scope under user@1000.service/app.slice gave illusion of continuity; user@1000 actually restarted at ~16:05 (new manager PID 262915, replacing 81142) and Claude Code resumed via SessionStart state-file, not process survival. This IS same class as BOB-124.

What happened

Operator report at 2026-08-19 14:11:37Z (mid-turn during autonomous post-wave sequence): “we had another logout !!!”. Sixth event in the same class (BOB-116 / BOB-120 / BOB-123 / BOB-124 preceded).

Distinction from prior incidents

FIRST INCIDENT WITH THE PHASE 1 AUDIT SEAM LIVE. Rules installed 2026-08-19 15:56 CEST (same kernel boot 0 that started 15:07):

```
-w /usr/bin/loginctl -p x -k logout_investigation
-w /usr/bin/systemctl -p x -k logout_investigation
-a always,exit -F arch=b64 -S kill,tkill,pidfd_send_signal -F
a1=0x9 -k sigkill_investigation
-a always,exit -F arch=b32 -S kill,tkill,pidfd_send_signal -F
a1=0x9 -k sigkill_investigation
-a always,exit -F arch=b64 -S tgkill -F a2=0x9 -k
sigkill_investigation
```

```
-a always,exit -F arch=b32 -S tkill -F a2=0x9 -k  
sigkill_investigation
```

Kernel audit is boot-persistent (survives user@1000 SIGKILL cascade because it lives in kernel space, not user session).

Attribution command (OPERATOR ACTION REQUIRED) — REV-2 corrected window

Phase-1 evidence (user manager PID transitions) narrows the event to ~16:05:03 (gdm-session-worker restart + graphical-session.target reached 16:05:11 under fresh manager PID 262915). Audit rules were installed 15:56 → 16:05 kill WAS in scope.

```
su -c  
    '/usr/sbin/ausearch -k sigkill_investigation --start 15:55  
    | head -200'  
su -c  
    '/usr/sbin/ausearch -k logout_investigation --start 15:55  
    | head -200'
```

What to look for in the SYSCALL records around 16:05: - pid=<X> — the SENDER PID - comm=<name> — sender program name (e.g. systemd, loginctl, oom_reaper, podman, pkill) - exe=<path> — full sender binary path - auid=<uid> — audit user id of sender - Paired OBJ_PID record: opid=<Y> — TARGET PID (should be user@1000 manager PID 81142)

Three honest interpretation paths per §11.4.6: 1. Real SIGKILL hits with clear initiator → attribute + fix at source 2. Empty (no hits in 15:55–16:10 window) → the kill mechanism bypasses ALL captured syscalls (kernel-internal OOM-kill would be one such class; different audit rule needed) 3. Overflow — audit backlog_limit too small; raise it before next incident

Paste the output — the SYSCALL records carry auid, pid, comm, exe for the initiator.

Systematic-debugging Phase 4.5 verdict — CATASTROPHIC

6 incidents in ~48h. This is the FIRST time we have kernel-side forensic capture. If ausearch returns:

- **Real SIGKILL(9) hits** — we finally attribute the initiator

- **Empty** — either audit backlog dropped events (buffer overflow) OR the “logout” was a different mechanism (session-lock, tty hangup, ForwardToWall unit reload — NOT a SIGKILL cascade). This would be a §11.4.6 correction of the entire investigation hypothesis.

Either outcome is progress.

Phase 4.5 STANDING RULE preserved

Per memory playbook: **do NOT author a 7th preventive gate**. Every unshipped gate raises the tower height without raising the ceiling (§11.4.250 heuristic-tower defect). Attribution first.

REV-3 Phase-1 breakthrough (2026-08-19 ~16:15) — system journal captured full cascade

System-wide journal (readable to my user for these messages) captured the exact event at **2026-08-19 16:04:54**:

```
16:02:26 audit[81557]: SYSCALL syscall=62 a1=9 pid=81557
ppid=81142 comm="dbus-daemon" exe="/usr/bin/dbus-daemon"
key="sigkill_investigation"
16:02:30 audit[81557]: SYSCALL syscall=62 a1=9 pid=81557
ppid=81142 comm="dbus-daemon" exe="/usr/bin/dbus-daemon"
key="sigkill_investigation"
16:04:54 gdm-password[81128]: pam_tcb(gdm-password:session):
Session closed for milosvasic
16:04:54 audit[81128]: AUDIT1106 op=PAM:session_close ... exe="/
usr/libexec/gdm-session-worker" terminal=/dev/tty2 ses=7
16:04:54 systemd[1]: user@1000.service: Main process exited,
code=killed, status=9/KILL
16:04:54 systemd[1]: user@1000.service: Killing process 83030
(gvfs-go-a-volume) with signal SIGKILL
16:04:54 systemd[1]: user@1000.service: Killing process 82548
(gsd-media-keys) with signal SIGKILL
16:04:54 systemd[1]: user@1000.service: Killing process 105594
(ssh) with signal SIGKILL
[... cascade of ~30 more processes ...]
```

Signature identity confirmed — same as BOB-116/120/123/124: PAM session_close on gdm-password precedes user@1000.service exited code=killed status=9/KILL. §11.4.6 Verdict: **BOB-125 is the 6th occurrence of the PAM/Linger contradiction (BOB-123 anchor mechanism).**

Memory pressure ruled OUT — PSI cumulative total=3575466 μ s = ~3.5s of full memory pressure across the entire 1h+ boot. Not OOM.

Attribution still pending — the SESSION_CLOSE was initiated by /usr/libexec/gdm-session-worker PID 81128, but WHO triggered gdm-worker to session-close (a loginctl kill-session, gnome-shell logout, X/Wayland session end, systemctl stop graphical-session.target, or something else) — that's the Phase-1 gap. Audit rules stay live for incident #7. NO 7th preventive gate authored per memory-playbook Phase 4.5 rule.

Two curious pre-cascade SIGKILL syscalls at 16:02:26 and 16:02:30 from dbus-daemon (PID 81557 under old user@1000 manager 81142). Same key sigkill_investigation. 2 minutes before the cascade. Could be dbus cleaning up dead peers OR could correlate with the terminator. Followup investigation candidate.

REV-4 REAL ROOT CAUSE FOUND + FIXED (2026-08-19 ~16:50)

BOB-126 (7th incident) captured the real root cause — this was NOT a “PAM/Linger contradiction” as prior 6 investigations claimed. All 6 prior incidents had the SAME true root cause, invisible until audit rules installed 15:56.

Root cause: tests/unit/merge_service/
test_deadline_tunable.py::test_deadline_hit_flag_true_when_readline_times_out created AsyncMock() without setting mock.pid as int.
Production code _search_public_tracker called
os.killpg(os.getpgid(proc.pid), signal.SIGKILL). Python's
MagicMock.__int__ defaults to 1, so: 1.int(MagicMock()) == 1 2.
os.getpgid(1) == 1 (init's process group) 3. os.killpg(1, SIGKILL) →
glibc → kill(-1, SIGKILL) = **SIGKILL every UID-1000 process on host**
4. contextlib.suppress(Exception) swallowed nothing — syscall
SUCCEEDED

Bug existed since 2026-04-24 (~4 months). Prior 6 forced-logout incidents were ALL this test being run under various conditions (my subagent sweeps, HelixQA autonomous mode, operator's IDE).

FIX — three-layer defense-in-depth: 1. **boba ad4b46a**: `search.py` `_search_public_tracker` int-guards `pid` + `pgid` before `killpg`. Both cleanup paths (deadline + exception handler) hardened. 2. **boba ad4b46a**: `test_deadline_tunable.py` sets `mock.pid=12345` explicitly + patches `os.killpg` and `os.getpgid` as belt-and-suspenders. New §11.4.115 RED-first regression test `test_bob126_regression_deadline_path_never_calls_killpg_with_pgid_le_1` asserts `killpg` is NEVER called with `pgid ≤ 1` even under the exact defect precondition. 3. **constitution 502586c**: NEW universal anchor §11.4.263 — **process-group signal-safety mandate**. Extends the fix to every project inheriting the constitution (Python/Go/Rust/Bash/C) with four-layer coverage per §11.4.4(b). Full text in `Constitution.md`; byte-identical compact mirrors in `CLAUDE/AGENTS/QWEN/GEMINI.md` (md5 542e30907eaa00ffb1949cd2fe528ba2). 4. **boba bf01cf3**: pointer bump.

Verification: 9/9 `test_deadline_tunable.py` PASS post-fix, including the new BOB-126 regression guard.

The 7-incident chain (BOB-116/120/123/124/125/126) is closed. No 8th incident should occur unless another codebase path calls `killpg` with unvalidated `pgid`.

Actions this session

- PAUSED post-wave sequence
- Filed this incident doc
- Waiting on operator's ausearch output before ANY further action
- Container fleet: proxy sub-fleet was bringing up when incident reported (my `./start.sh -p` had just launched at 16:11:06 — proxy sub-fleet restart may or may not correlate)

Cross-references

- `docs/incidents/2026-08-19-5th-forced-logout.md` (BOB-124)
- Memory playbook: `~/.claude-claude4/.../memory/forced_logout_incidents.md`

- docs/QA_DISCOVERY_LEDGER.md Rev 11 (BOB-124 escape entry)